

Theory of Operation

1053 Emulator for IBM 1130

Summary

This project created a box that plugs into the IBM 1130 in the place of the 1053 Console Printer that is built into the IBM 1130 System. The box connects via a USB serial cable to a Windows PC running the Putty emulator program.

The Putty program displays all the output exactly as it would appear on the paper in the 1053 printer. It also saves the output in a log file if hardcopy is desired.

The 1053 Emulator is designed to mimic all the behaviors and output of the console printer. The box that contains the emulator provides a three digit display to show the column where the carrier is sitting, buttons to request Return, Tab and Space, and buttons to set or clear a tab stop at the current column.

The other actions that a user could perform on the real 1053 are changing the left and right margins, or switching the golf-ball type element. The margins are changed by typing a command from Putty to the 1053 Emulator

The usual type element for the 1130 is number 969, but ball number 988 can be installed to use the 1130 to run the APL (A Programming Language) system. Another command can be entered through Putty to switch between the type balls.

Theory of Operation

1053 Emulator for IBM 1130

The 1053 Console Printer

The console printer is a print only I/O Selectric typewriter. Solenoids are added to the Selectric typewriter mechanism to drive the mechanism. A set of solenoids trigger the following actions:

- Space
- Backspace
- Carrier return/line feed
- Line Feed
- Tab

Another set of solenoids operate the character selection logic that determine how the ball tilts and rotates to pick one of 44 positions on the gold ball hemisphere. There are two that choose among the four rows of characters - Tilt 1 and Tilt 2. Four select the rotation among 11 columns on the ball - R1, R2, R2A and R5. A final solenoid called Aux is triggered with every request to type a character.

Another two solenoids control the shift mechanism, which spins the golf ball 180 degrees between the two hemispheres. On an office typewriter these are the upper case and lower case characters.

A final two solenoids select the ribbon color, choosing between typing on the upper half of the ribbon or the lower half - as the ribbon is bicolor with red and black ink

Microswitches are also added to the Selectric mechanism to provide feedback to the 1130 about the state of the typewriter. These combine to generate two key feedback signals that indicate when it is safe to request new printing or movement commands as well as when long operations like carrier return have completed.

Theory of Operation

1053 Emulator for IBM 1130

The Selectric mechanism is motor driven but uses clutches to fire off cycles when it is printing or moving. A print cycle is a 180 degree rotation of the main shaft which occurs when the character selection solenoids activate, moving the ball and flinging it forward to strike the paper through the ribbon.

In addition to typing a character, a 180 degree spin of the operational shaft cycle is used to operate the shift mechanism. When the next character to type is on the other hemisphere, one cycle is required to shift and then a subsequent print cycle does the actual printing of the carrier.

Movements that have a fixed time to operate also take a 180 degree turn of the operational shaft to complete. These are space, backspace and starting a tab movement.

Line feed requires a 360 degree rotation of the operational shaft, thus it is considered a long operation. Both tab movement and a carrier return require a variable amount of time to complete, depending upon the starting and ending column for the movement. These are also long operations.

-TWR CB RESPONSE is activated during the short operations and indicates the times when the mechanism is busy and cannot safely accept another request for movement or to type another character. A microswitch on the print cycle shaft opens during part of the rotation. Microswitches on the rear connected to the space and to the backspace mechanisms open while those are occurring. Finally, a microswitch on the mechanism that latches the carrier free so that it can begin a tab movement will signal that the machine is busy starting to move. ‘

+TWR CRLFT INLK is activated during the long operations. Once the tab has begun and the carrier is latched free of the teeth so it can move, a microswitch on the latch keeps this signal active until the carrier rams into a tab stop which resets the latch as it stops. A microswitch on the

Theory of Operation

1053 Emulator for IBM 1130

carrier return mechanism latch is turned on when the mechanism latches the return, which drives the carrier to the left under power. When the carrier reaches the left margin, this disconnects the latch and releases the microswitch.

Two additional microswitches activate the signal at the start when the solenoid that requests line feed, carrier return or tab is first triggered, which blocks further requests from the 1130 during the early part of the long operations, until those latches on the return and on the tab mechanisms are active.

Two additional feedback signals are produced. One indicates that there is paper loaded into the typewriter, with a microswitch that the paper holds down. This is *-TWR END FORMS*. Lastly, a microswitch on the right margin stop is activated when the carrier reaches the right margin position, generating *+TWR END LINE* to trigger an automatic command from the 1130 to return the carrier and feed a line.

The typewriter mechanism can print at a maximum rate of about 15.6 characters per second if the request for each character is presented by activating the solenoids before the print cycle rotation has come to a stop.

The feedback signal *-TWR CB RESPONSE* allows the 1130 to send the next request as soon as it is safe to do so, allowing the machine to take continuous rotations without having to start and stop at the 180 degree mark.

Whenever the next character is on a different hemisphere from the one before, a shift cycle has to occur which slows down the machine. The maximum rate can only be achieved with a stream of characters all on the same hemisphere.

Theory of Operation

1053 Emulator for IBM 1130

The printer uses paper forms with pin feed holes on the two edges, with the typewriter tractor mechanism moving the forms through the machine reliably. Boxes of paper forms in this size are no longer available.

The line width is 120 columns, although the user can adjust the left and right margins to a narrower range. An indicator on the front of the printer shows the current column where the carrier is sitting.

Like any Selectric based typewriter, it provides tab stops to allow the software to move the carrier to predetermined columns. These are set or cleared by manually moving the carrier to the desired column then turning the lever on the front of the printer to set or clear the tab stop.

The front of the printer also has three buttons that allow the user to trigger a carrier return/line feed, tab the carrier to the next tab stop, or space the carrier one column to the right.

The front of the printer also hosts a set of 16 toggle switches, named the Console Entry Switches. These are not related to the console printer at all, other than being mounted on the printer's front panel

Theory of Operation

1053 Emulator for IBM 1130

The 1053 Emulator hardware

The emulator is constructed around an Arduino Mega 2560 controller board. It detects the signals from the 1130 that fire the various solenoids and it generates the four feedback signals to the 1130.

The 1053 has 48V applied to one side of each solenoid and the other side is connected to the controller logic inside the 1130. The controller logic is like an open collector gate - it either grounds the signal or leaves it untouched.

This is fortunate, because the Arduino can connect to the same logic gates but use a pullup resistor to +5V instead of dealing with 48V. When the controller logic pulls a line to ground, the Arduino sees its value drop from logic high to logic low. No interface circuitry is required for all the solenoid signals.

The feedback signals, on the other hand, must deliver a higher voltage to a switch debouncer circuit in the 1130 controller logic. Three of the signals have +12V running through the microswitches, but the *-TWR CB RESPONSE* microswitches are fed with 48V.

The Arduino cannot handle these voltages, so I have a switching board that is driven by a 0-5V Arduino output and controls a MOSFET switch which acts almost exactly like the microswitches. The only different is that there is no switch bounce. Four circuits switch the four feedback signals.

Theory of Operation

1053 Emulator for IBM 1130

The box also contains five pushbuttons hooked to the Arduino, to activate the Tab Set, Tab Clear, Return, Tab and Space functions. Finally, the box has a board that drives three seven segment displays, using BCD to 7 Segment decoder chips (74LS47) to produce the digits.

Cables run from the box to connect into the 1130. One cable brings signals to an SMS paddle card (PF1), another cable brings signals to an SMS socket (PF2). The third is an SMS power paddle card that would deliver multiple voltages to the 1053. We ignore the 120V that the typewriter needs for its motor, but do use the +48V, +12V and ground lines.

Theory of Operation

1053 Emulator for IBM 1130

High level view of the Arduino code

The main loop first checks to see if data is arriving over the USB serial link, which it assembles until we reach eight bytes with a newline at the end. This is interpreted and if it is validly formatted, action is taken.

The valid incoming data can be a TS ###, TC ###, LM ###, or RM ### to set or clear tab stops or the left or right margin to column ###. Two other valid statements are TABS = and TYPE # which display all columns with tabs set or switches between type balls A or N.

All of the incoming signals from the 1130 controller logic are configured as inputs with a pullup resistor to 5V. The controller logic will pull the signal to ground to assert the request to activate a solenoid in the 1053.

Any request to type a character involves some number of selection solenoids being activated but always includes the AUX signal. Just as with the 1053 mechanism, the first selection signal that is asserted triggers a 65 millisecond print cycle.

The code waits until we are 5 ms into the print cycle so that other selection solenoid requests can arrive, as they might lag slightly from the first signal to be detected. At this time, the combination of selection solenoid requests is interpreted to determine what character on the type ball is to be printed.

Theory of Operation

1053 Emulator for IBM 1130

The selection solenoids T2, T1, R1, R2, R2A, R5 and AUX are used to look up an index in a tilt-rotate table. That index points to which of the 44 positions - four rows of tilt 0-3, eleven columns of rotate -5 to +5 - has the intended character.

Depending on which typeball (969 - normal, or 988 - APL) is installed and whether the ball is turned to the upper case or lower case hemisphere, we look in one of four character tables to pick up the unicode character.

At specific times in the 65 ms print cycle, the feedback signal -*TWR CB RESPONSE* is asserted and then deasserted. The unicode character we picked up from the tables is stored in a line buffer at the current column position of the typeball carrier.

When we begin a new line we zero out a buffer that has entries for all 120 columns that can be typed upon by the 1053, with five entries per column to store characters that were typed there.

This buffer exists to support overtyping, where characters are typed in a column more than once. If the second and other subsequent characters typed in the same column are different, then the 'ink' from each is left on the paper which forms a composite impression.

Theory of Operation

1053 Emulator for IBM 1130

Composite impressions can be used to obscure sensitive data, for example, but they can also be used to form a character that does not exist on the type ball. With only 88 positions on a type ball, 36 of which are necessary just to cover the alphabet and numeric digits, composite characters may be needed to support a broad character set requirement for some software that exceeds the capacity of a typeball.

Thus, every time we print at a column, the character is added to the buffer and then we ask the Putty terminal program to combine the pixels to form the composite. This is done by sending the characters with a special unicode character Zero Width Join (ZWJ) in between the characters. If more than five are overtyped in a column, we just keep the first four and the last, dropping others.

The way that an overwrite is accomplished on the 1053 is to use the backspace movement to shift the typeball carrier leftward to lower numbered columns up to the minimum of the left margin column. Then the additional character is typed.

Terminal emulator programs like Putty will move backwards when receiving a backspace character and erase what was on the screen at the new column position. With paper and the 1053, the type is not erased when we backspace. That is why we have to retype the characters at the new column to put them back on the screen.

Theory of Operation

1053 Emulator for IBM 1130

When a space movement occurs with the 1053, it just moves one column to the right, leaving the paper underneath undisturbed. In the emulator, we have to print a space character the first time we reach a column moving from left to right, but if we had backspaced and then space forward, we do NOT want to blank out what was previously there.

The code checks the line buffer to see if anything had been printed in the column; if so, we just repeat typing the data in the buffer while we move the cursor one column forward.

This supports the ability to edit a line by moving back and forward using backspace and space movement commands, leaving what is viewed on the screen appearing just as they were. Any typing that happens in a column where we previously typed is used to form a composite on the screen.

The movement requests space, backspace and tab will also kick off a print cycle of 65 ms, but without the AUX solenoid being fired, we don't have a character to 'type' on the screen. We change the column where the typeball sits, retyping anything in columns we pass over, if characters had been typed there.

The code emits space and backspace characters to the terminal emulator program, or retypes a column, so that the screen matches what we would see on the paper on a 1053.

Theory of Operation

1053 Emulator for IBM 1130

We use a fast search method to look at the byte array, 120 bytes for the 120 columns, searching for the first non-zero byte. That is the location of the next tab stop. We space over to that column to finish the tab movement or retype from the buffer as appropriate.

Solenoid assertions for carrier return/line feed, tab, or line feed also trigger a long cycle, which typically extends past the 65ms of the primary print cycle. At fixed times during the long cycle, the *+TWR CRLFT INLK* feedback signal is asserted and then deasserted.

The time it will take to finish the long cycle depends on the operation and also both the current column number and the destination column number. A carrier return from the right side takes longer than if it is issued only ten columns to the right of the margin. A tab duration depends on how many columns we have to pass from the current column to the first one that has a tab stop set. Line feed takes a fixed amount of time to occur.

This is calculated when we start the long cycle and used to control the deassertion of the *+TWR CRLFT INLK* feedback signal

Activations for the Shift to Upper or Shift to Lower solenoids trigger a 65 ms print cycle, but without the AUX solenoid asserted, nothing is typed. We simply record the hemisphere that is reached as this is used during the character table lookups when printing.

Theory of Operation

1053 Emulator for IBM 1130

Asserting the Shift to Red or Shift to Black solenoids does NOT trigger a print cycle or provide any feedback signals. It causes the code to send an ANSI Color sequence to the Putty terminal program which changes the text color to red or black.

On the 1053, the user slides levers to set the left and right margins. The left margin is the point where the typeball carrier stops when doing a carrier return. It is the point beyond which we cannot backspace. The right margin is where the *+TWR END LINE* feedback signal is asserted if the carrier reaches that column.

The controller logic in the 1130 will immediately trigger a carrier return/line feed cycle by asserting that solenoid when the end of line feedback signal is on. The controller marks the typewriter busy until the return has finished based on the feedback signal *+TWR CRLFT INLK* going low.

However, if the user somehow manages to drive print requests when we have reached the right margin, the code does not advance the column, it backspaces and erases whatever character we tried to type. This matches how the 1053 behaves in the same circumstances.

The last feedback signal from the 1053 that we control is *-TWR END FORMS* which is asserted low if the paper has run out on the 1053. The controller logic will not type or move the carrier when this is asserted and a FORMS CHECK lamp on the console of the IBM 1130 will illuminate..

Theory of Operation

1053 Emulator for IBM 1130

If the 1053 Emulator is powered down, the 1130 system sees the *-TWR END FORMS* signal as asserted and lights the FORMS CHECK lamp. During the setup part of the code initialization, we deassert this, extinguishing the lamp.

Theory of Operation

1053 Emulator for IBM 1130

Special techniques used in the code

We make use of direct port register access rather than using the slower `digitalRead` and `digitalWrite` statements. Refer to a port map for the Arduino Mega 2560 to see which registers and bits of the Microchips ATmega2560 processor chip are associated with each of the Arduino pins.

I used the traditional Arduino statements to set the input or output mode for the pins, but then I directly access the ports to read or write. For a read, I examine the specific bit of the associated port, using macros for readability:

```
#define readAUX  ! (PINC & (1<<3))
```

For writes, I either set or clear the specific bit in the associated register to set the Arduino pin high or low. I set up macros to do this, for example:

```
#define highEOL  PORTE |= (1<<5)
#define lowEOL   PORTE &= ~(1<<5)
```

Rather than looping through an array of 120 locations looking for the next tab stop, I used a fast search that moves through a byte array by fetching doublewords at a time until the doubleword is non-zero, then determining which of the bytes inside has the first non-zero value.

I do make use of character pointer arithmetic to index into arrays, thus `array[index]` or `array+index` is equivalent.